

Bernoulli Maximum-Likelihood Estimation, Part II: The Practical Solution — Turning Estimation into Optimization

Yuval Lavie

August 11, 2026

Abstract

Part I (`mle_theoretical.tex`) solved the Bernoulli estimation problem the statistician's way: maximize the likelihood, in closed form. This part performs the machine-learning translation — likelihood $\rightarrow$ loss — and solves the *same* problem the way we must when no closed form exists: gradient descent with a hand-derived gradient, stochastic gradient descent from one sample at a time, and SGD with the gradient computed by PyTorch's autograd. Two companion scripts share Part I's dataset: `mle_practical_pure.py` implements GD and SGD by hand and *animates* the estimator descending the loss landscape alongside its loss progression; `mle_practical_pytorch.py` drives autograd through the identical sample schedule and verifies the trajectories coincide to machine precision — a proof by computation that autograd performs exactly our chain rule.

1 From likelihood to loss

Part I ended with the log-likelihood $\ell(p) = m \log p + (n - m) \log(1 - p)$ and its maximizer $\hat{p} = m/n = \bar{x}$. Machine learning prefers to *minimize losses*, and two cosmetic transformations get us there — neither moves the optimum. Dividing by the constant $n > 0$ rescales the function but does not move its argmax (and makes the scale independent of dataset size). Negating flips maximization into minimization. With $\bar{x} = m/n$, the resulting *average negative log-likelihood* (NLL) is

$$\mathcal{L}(p) = -\frac{1}{n} \ell(p) = -\bar{x} \log p - (1 - \bar{x}) \log(1 - p), \quad (1)$$

and the chain of equivalences

$$\arg \max_p L(p) = \arg \max_p \ell(p) = \arg \min_p \mathcal{L}(p) \quad (2)$$

says minimizing this loss *is* maximum likelihood. Written as an average of per-sample terms,

$$\mathcal{L}(p) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_i(p), \quad \mathcal{L}_i(p) = -x_i \log p - (1 - x_i) \log(1 - p), \quad (3)$$

which is the *cross-entropy* loss — and the sum-of-per-sample structure is precisely what SGD will exploit.

2 One coordinate change — not a new objective

Gradient methods want an unconstrained variable, but p lives in $(0, 1)$. So we evaluate the *same* NLL (1) at

$$p = \sigma(t) = \frac{1}{1 + e^{-t}}, \quad t \in \mathbb{R}, \quad F(t) = \mathcal{L}(\sigma(t)). \quad (4)$$

Nothing about the target function changes; we have only composed it with a smooth bijection of the search space. (By the *invariance* of maximum likelihood — Part I’s closing remark — the optimum maps back: $\hat{p} = \sigma(\hat{t})$.)

The gradient, by the chain rule. Using $\sigma'(t) = \sigma(t)(1 - \sigma(t))$ and writing $s = \sigma(t)$ for the moment,

$$F'(t) = \mathcal{L}'(s) \sigma'(t) = \left[-\frac{\bar{x}}{s} + \frac{1 - \bar{x}}{1 - s} \right] s(1 - s) = -\bar{x}(1 - s) + (1 - \bar{x})s = s - \bar{x},$$

hence

$$\boxed{F'(t) = \sigma(t) - \bar{x}} \quad (5)$$

Every term cancelled except the residual: current estimate minus sample mean. Moreover $F''(t) = \sigma'(t) > 0$, so F is convex in t — one basin.

3 Route 1: gradient descent

GD iterates the full gradient, every step:

$$t \leftarrow t - \eta (\sigma(t) - \bar{x}). \quad (6)$$

This is deterministic: with a sane step size it converges monotonically to the unique fixed point $\sigma(t^*) = \bar{x}$ — Part I’s closed form, now reached by iteration. For this toy problem the “full gradient” is free ($\bar{x}$ is precomputed); in real models it costs a pass over the entire dataset per step, which is what motivates the next route.

4 Route 2: stochastic gradient descent

SGD replaces the full gradient (5) with a *one-sample estimate*: draw an index i and use the gradient of the per-sample term (3) through the same coordinate change,

$$g_i(t) = \sigma(t) - x_i. \quad (7)$$

It is unbiased — averaging over a uniformly drawn i , $\mathbb{E}_i[g_i(t)] = \sigma(t) - \bar{x} = F'(t)$ — so each cheap step points in the right direction *on average*. The script sweeps reshuffled passes over the data, updating $t \leftarrow t - \eta g_i(t)$ one sample at a time.

The price of stochasticity, and Polyak–Ruppert averaging. With a constant step size the SGD iterate never converges: it reaches a stationary *distribution*, a noise ball around the optimum whose radius grows with η and the per-sample variance. The estimator is therefore taken as the average of the final pass’s iterates, $\hat{p}_{\text{SGD}} = \frac{1}{K} \sum_k \sigma(t_k)$, which cancels the hovering and lands within a few 10^{-3} of $\bar{x}$.

Remark 1 (Mini-batches, later). *Averaging g_i over a small batch B gives $\sigma(t) - \bar{x}_B$: the same expectation with variance reduced by $|B|$. Everything in this note goes through unchanged; batching is an engineering trade between noise and cost per step, not new mathematics.*

5 Route 3: SGD with autograd

PyTorch’s `binary_cross_entropy_with_logits(t, x_i)` evaluates exactly the one-sample NLL at $p = \sigma(t)$, i.e. the composition $F_i(t) = \mathcal{L}_i(\sigma(t))$ of (3) with (4). Calling `loss.backward()` applies reverse-mode automatic differentiation — a mechanized chain rule — so it must produce `t.grad` $= \sigma(t) - x_i$, identically (7). `torch.optim.SGD` then performs the same update $t \leftarrow t - \eta \mathbf{t.grad}$. Autograd changes who does the calculus, not the mathematics.

Proof by computation. The script drives the hand-written SGD and the autograd SGD through the *identical* sample schedule, the same η , and the same initialization $t_0 = 0$ (in `float64` on both sides). Under those conditions the two trajectories can differ only if the gradients differ; the script asserts $\max_k |\sigma(t_k^{\text{hand}}) - \sigma(t_k^{\text{autograd}})| < 10^{-10}$, and measures the gap to be exactly 0.0 over all 15,001 iterates — an end-to-end check that (7) is precisely what autograd computes.

6 Experiment

With $p = 0.3$ and $n = 5000$ (seed 7, Part I’s dataset): GD ($\eta = 1$, 150 steps) lands 5.6×10^{-17} from the closed form; SGD ($\eta = 0.1$, 3 epochs), Polyak-averaged over the last pass, lands within 1.5×10^{-4} ; and the two SGD trajectories coincide exactly.

`mle_practical_pure.py` plays the animation, three synchronized views of the same descent: the loss landscape (1) with GD’s bead gliding down while SGD’s bead jitters into its noise ball; the estimate-versus-time curves approaching the closed-form line; and the *loss progression* — the suboptimality gap $\mathcal{L}(p_k) - \mathcal{L}(\hat{p})$ on a log scale, where GD’s geometric convergence is a straight plunge and SGD’s noise floor is a plateau. `mle_practical_pytorch.py` animates the proof by computation: the hand and autograd trajectories drawing simultaneously and never separating, beside their per-step gap pinned to zero on a log scale.

Three routes, one destination: the loss surface — not the optimizer — decides where estimation lands; the optimizer only decides how you travel and what noise you accept along the way.